СЕМИНАР 6 · ВВЕДЕНИЕ В БАЗЫ ДАННЫХ

JOIN и операции над множествами

Зачем соединять таблицы

Часто нужно обращаться сразу к нескольким таблицам в запросе. Например, добавить к платежу имя клиента из другой таблицы по его ключу.

Таблица платежей		**Таблица клиентов**		**Итоговая выборка**
ID платежа Сумма ID клиента	JOIN	ID клиента Город проживания Имя	=	ID платежа Сумма ID клиента Имя клиента

При помощи соединения можно сопоставить строки одной таблицы со строками другой. В SQL для этого существует предложение JOIN.

JOIN: синтаксис

СИНТАКСИС: условие после ON

```
SELECT {поля, которые выводим}
FROM {таблица1}
  JOIN {таблица2}
    ON таблица1.поле_соединения =
       таблица2.поле_соединения
```

СИНТАКСИС: одинаковые имена полей — USING

```
SELECT {поля, которые выводим}
FROM {таблица1}
  JOIN {таблица2}
    USING (поле_соединения)
```

- Предложение JOIN записывается после FROM
- После ON указывается условие соединения — по каким полям сопоставляются строки
- Если поля соединения в двух таблицах называются одинаково, вместо ON можно использовать USING

INNER JOIN: внутреннее соединение

СИНТАКСИС

```
SELECT {поля, которые выводим}
FROM {таблица1}
  INNER JOIN {таблица2}
    ON таблица1.поле_соединения =
       таблица2.поле_соединения
```

ПРИМЕР

Определить профессии всех сотрудников.

```
SELECT *
FROM "Таблица A" a
  INNER JOIN "Таблица B" b
    ON a.client_id = b.client_id
```

Таблица A

client_id	name	gender_cd
1	Макс	M
2	Наташа	F
3	Никита	M

Таблица B

client_id	profession
4	Продакт-менеджер
2	Аналитик
3	Инженер данных

Результат

a.client_id	a.name	gender_cd	b.client_id	b.profession
2	Наташа	F	2	Аналитик
3	Никита	M	3	Инженер данных

INNER JOIN: отбор строк и столбцы

ПРИМЕР

Определить профессию всех клиенток.

```
SELECT *
FROM "Таблица A" a
  INNER JOIN "Таблица B" b
    ON a.client_id = b.client_id
WHERE gender_cd = 'F'
```

Результат

a.client_id	a.name	gender_cd	b.client_id	b.profession
2	Наташа	F	2	Аналитик

- Если записать просто JOIN, будет выполнен INNER JOIN
- Условий после ON может быть несколько — через AND / OR
- A INNER JOIN B и B INNER JOIN A эквивалентны
- SELECT * выводит столбцы обеих таблиц — лучше перечислять только нужные
- Столбец, который есть в обеих таблицах, пишется с алиасом: a.client_id — иначе ошибка

CROSS JOIN: перекрёстное соединение

СИНТАКСИС

```
SELECT *
FROM "Таблица A" a
  CROSS JOIN "Таблица B" b
-- ИЛИ
SELECT *
FROM "Таблица A", "Таблица B"
```

ПРИМЕР

Соединим с помощью CROSS JOIN таблицу дат и клиентов.

```
SELECT *
FROM таблица_дат
  CROSS JOIN таблица_клиентов
```

Таблица дат

Дата
2024-01-01
2024-02-01
2024-03-01

Таблица клиентов

Клиент
Вася
Петя
Егор

Каждая строка с каждой: 3 × 3 = 9 строк.

Условие ON не пишется.

Результат

Дата	Клиент
2024-01-01	Вася
2024-01-01	Петя
2024-01-01	Егор
2024-02-01	Вася
2024-02-01	Петя
2024-02-01	Егор
2024-03-01	Вася
2024-03-01	Петя
2024-03-01	Егор

LEFT JOIN: всё из первой таблицы

СИНТАКСИС

```
SELECT {поля из таблиц}
FROM {первая таблица} a
  LEFT JOIN {вторая таблица} b
    ON a.{поле соединения} =
       b.{поле соединения}
```

Сохраняем всё из первой таблицы, добавляем данные из второй. Если во второй ничего не нашлось — NULL.

ПРИМЕР

```
SELECT *
FROM "Таблица A" a
  LEFT JOIN "Таблица B" b
    ON a.client_id = b.client_id
```

Таблица A

client_id	name	gender_cd
1	Макс	M
2	Наташа	F
3	Никита	M

Таблица B

client_id	profession
4	Продакт-менеджер
2	Аналитик
3	Инженер данных

Результат

a.client_id	a.name	gender_cd	b.client_id	b.profession
1	Макс	M	NULL	NULL
2	Наташа	F	2	Аналитик
3	Никита	M	3	Инженер данных

RIGHT JOIN: всё из второй таблицы

СИНТАКСИС

```
SELECT {поля из таблиц}
FROM {первая таблица} a
  RIGHT JOIN {вторая таблица} b
    ON a.{поле соединения} =
       b.{поле соединения}
```

Выводим все данные из второй таблицы, добавляем, где есть, данные из первой, остальное — NULL.

ПРИМЕР

```
SELECT *
FROM "Таблица A" a
  RIGHT JOIN "Таблица B" b
    ON a.client_id = b.client_id
```

Таблица A

client_id	name	gender_cd
1	Макс	M
2	Наташа	F
3	Никита	M

Таблица B

client_id	profession
4	Продакт-менеджер
2	Аналитик
3	Инженер данных

Результат

a.client_id	a.name	gender_cd	b.client_id	b.profession
NULL	NULL	NULL	4	Продакт-менеджер
2	Наташа	F	2	Аналитик
3	Никита	M	3	Инженер данных

FULL JOIN: всё из обеих таблиц

СИНТАКСИС

```
SELECT {поля из таблиц}
FROM {первая таблица} a
  FULL JOIN {вторая таблица} b
    ON a.{поле соединения} =
       b.{поле соединения}
```

Выводит все строки как первой, так и второй таблицы. Если чего-то не хватает — NULL.

ПРИМЕР

```
SELECT *
FROM "Таблица A" a
  FULL JOIN "Таблица B" b
    ON a.client_id = b.client_id
```

Таблица A

client_id	name	gender_cd
1	Макс	M
2	Наташа	F
3	Никита	M

Таблица B

client_id	profession
4	Продакт-менеджер
2	Аналитик
3	Инженер данных

Результат

a.client_id	a.name	gender_cd	b.client_id	b.profession
NULL	NULL	NULL	4	Продакт-менеджер
2	Наташа	F	2	Аналитик
3	Никита	M	3	Инженер данных
1	Макс	M	NULL	NULL

LEFT или RIGHT: дело в порядке таблиц

ПРИМЕР

Вывести всех клиентов-мужчин и, если есть, добавить информацию об их профессии.

```
SELECT *
FROM "Таблица A" a
  LEFT JOIN "Таблица B" b
    ON a.client_id = b.client_id
WHERE gender_cd = 'M'
```

```
SELECT *
FROM "Таблица B" b
  RIGHT JOIN "Таблица A" a
    ON a.client_id = b.client_id
WHERE gender_cd = 'M'
```

Результат

a.client_id	a.name	gender_cd	b.client_id	b.profession
1	Макс	M	NULL	NULL
3	Никита	M	3	Инженер данных

Результаты эквивалентны: сохраняется таблица A — она первая в LEFT JOIN и вторая в RIGHT JOIN.

INNER JOIN и FULL JOIN от перестановки таблиц не меняются.

Строки без пары: LEFT JOIN и IS NULL

ПРИМЕР

Вывести из table1 ID клиентов, чью профессию мы не знаем.

Шаг 1. Профессия нужна для всех пользователей table1 — значит, LEFT JOIN.

Шаг 2. У пользователей без профессии столбцы table2 — NULL.

Шаг 3. Оставляем такие строки условием в WHERE.

```
SELECT t1.user_id
FROM table1 AS t1
  LEFT JOIN table2 AS t2
    ON t1.user_id = t2.user_id
WHERE t2.profession IS NULL
```

Результат: user_id = 4

table1

user_id	name
1	Андрей
2	Артём
3	Мария
4	Георгий

table2

user_id	profession
1	Аналитик
2	Дизайнер
3	Преподаватель
5	Таксист

Шаги 1–2: результат LEFT JOIN

user_id	name	profession
1	Андрей	Аналитик
2	Артём	Дизайнер
3	Мария	Преподаватель
4	Георгий	NULL

Условия соединения

Условие	Как пишется	Что получится
По равенству	`ON t1.user_id = t2.user_id`	пары строк с равными значениями
По неравенству	`ON t1.salary_need <= t2.max_salary`	все пары, для которых сравнение верно
Всегда верное	`ON 1=1`	каждая строка с каждой: 4 × 4 = 16 строк
Всегда ложное	`ON 1=0`	0 строк
Несколько условий	`ON t1.user_id = t2.user_id` `AND t2.profession = 'Аналитик'`	условия через AND / OR, как в WHERE

INNER JOIN (JOIN) — это CROSS JOIN с фильтрами, указанными после ON.

JOIN по неравенству

ПРИМЕР

Определить, какие вакансии подходят каждому клиенту.

```
SELECT *
FROM table1 AS t1
  JOIN table2 AS t2
    ON t1.salary_need <= t2.max_salary
```

table1

user	salary_need
Андрей	300 000
Артём	100 000
Мария	150 000
Георгий	50 000

table2

profession	max_salary
Аналитик	250 000
Дизайнер	90 000
Преподаватель	60 000
Таксист	30 000

Результат

user	salary_need	profession	max_salary
Артём	100 000	Аналитик	250 000
Мария	150 000	Аналитик	250 000
Георгий	50 000	Аналитик	250 000
Георгий	50 000	Дизайнер	90 000
Георгий	50 000	Преподаватель	60 000

Сравнение по неравенству — очень затратная операция: по возможности не использовать его для соединения таблиц.

Условие в ON или в WHERE

INNER JOIN, только аналитики: условие в ON

```
SELECT t1.*, t2.profession
FROM table1 t1
  INNER JOIN table2 t2
    ON t1.user_id = t2.user_id
      AND t2.profession = 'Аналитик'
```

INNER JOIN: то же условие в WHERE

```
SELECT t1.*, t2.profession
FROM table1 t1
  INNER JOIN table2 t2
    ON t1.user_id = t2.user_id
WHERE t2.profession = 'Аналитик'
```

Результат одинаковый: 1 | Андрей | Аналитик

LEFT JOIN, без профессии: условие в WHERE

```
SELECT t1.user_id
FROM table1 AS t1
  LEFT JOIN table2 AS t2
    ON t1.user_id = t2.user_id
WHERE t2.profession IS NULL
```

Результат: 4

LEFT JOIN: то же условие в ON

```
SELECT t1.user_id
FROM table1 AS t1
  LEFT JOIN table2 AS t2
    ON t1.user_id = t2.user_id
      AND t2.profession IS NULL
```

Результат: 1, 2, 3, 4 — не то же самое

Многократные JOIN

Многократные JOIN объединяют данные из нескольких таблиц в один результирующий набор.
Когда использовать:

- для анализа взаимосвязей между данными из нескольких таблиц — например, продаж по регионам и продуктам
- чтобы создавать отчёты из различных источников — например, отчёт о клиентах с их контактами, историей заказов и данными о доставке

СИНТАКСИС

```
SELECT {поля, которые выводим}
FROM {таблица1}
  (INNER / LEFT / RIGHT) JOIN {таблица2}
    ON таблица1.поле_соединения = таблица2.поле_соединения
  (INNER / LEFT / RIGHT) JOIN {таблица3}
    ON таблица1.поле_соединения = таблица3.поле_соединения
  (INNER / LEFT / RIGHT) JOIN {таблица4}
    ON таблица2.поле_соединения = таблица4.поле_соединения
```

Одна и та же таблица в JOIN несколько раз

СИНТАКСИС

```
SELECT *
FROM "Таблица A" a
  (INNER / LEFT / RIGHT) JOIN "Таблица A" b
    ON {условие соединения}
```

ПРИМЕР

Вывести рядом с именем сотрудника имя его руководителя. Информация о руководителе хранится в той же таблице.

```
SELECT e.name AS emp_name,
       m.name AS manager_name
FROM emp AS e
  JOIN emp AS m
    ON e.manager_id = m.emp_id
```

emp

emp_id	name	manager_id
1	Андрей	3
2	Артём	3
3	Мария	4
4	Георгий	NULL

Результат

emp_name	manager_name
Андрей	Мария
Артём	Мария
Мария	Георгий

Иногда называется SELF JOIN. Это не отдельное предложение SQL: таблица связывается сама с собой под разными алиасами.

ВАЖНОЕ ПРО JOIN

Порядок выполнения запроса с JOIN



- JOIN выполняется сразу после FROM: сначала таблицы соединяются, потом из результата отбираются строки
- WHERE, GROUP BY и агрегаты работают уже с соединённой таблицей

Все виды соединения

Соединение	Результат
`A INNER JOIN B`	К строкам в таблице A присоединит строки из таблицы B. Если соответствие не было найдено, то строки из A (или из B) не будут выведены
`A LEFT OUTER JOIN B`	Выведет все строки таблицы A, к ним добавит строки из таблицы B (где это соответствие было найдено). Если соответствие не было найдено, то заполнит все столбцы из таблицы B пропусками
`A RIGHT OUTER JOIN B`	Выведет все строки таблицы B, к ним добавит строки из таблицы A (где это соответствие было найдено). Если соответствие не было найдено, то заполнит все столбцы из таблицы A пропусками
`A FULL OUTER JOIN B`	Выведет все строки из A и B. Те, которым не нашли соответствие, заполнит пропусками
`A CROSS JOIN B`	Все возможные комбинации строк из A и B

Операции над множествами: когда нужны

- объединить результаты из нескольких таблиц с одинаковой структурой
- найти общие элементы двух наборов
- найти различия между наборами
- подготовить «сводку» из разных источников без джойнов — когда связи между таблицами нет или она не нужна

Города

Тип	Название	Население
Город	Москва	13 000 000
Город	Санкт-Петербург	5 000 000

+

Страны

Тип	Название	Население
Страна	Россия	130 000 000
Страна	Бразилия	140 000 000

=

Одна таблица: строки под строками

Тип	Название	Население
Город	Москва	13 000 000
Город	Санкт-Петербург	5 000 000
Страна	Россия	130 000 000
Страна	Бразилия	140 000 000

Для этих ситуаций JOIN не подходит: он дописывает столбцы, а здесь нужно добавить строки.

UNION, INTERSECT, EXCEPT

Операция	Что возвращает	Вариант с ALL
`UNION`	объединение: уникальные строки из двух наборов	UNION ALL — объединение с сохранением дублей
`INTERSECT`	пересечение: уникальные строки, которые есть в обеих выборках	INTERSECT ALL — учитывает кратность строк
`EXCEPT`	разность: уникальные строки первой выборки, которых нет во второй	EXCEPT ALL — учитывает кратность строк

БАЗОВЫЙ СИНТАКСИС

```
SELECT ...      -- множество 1
FROM ...
[WHERE ...] [GROUP BY ...] [HAVING ...]
[ORDER BY ... LIMIT ...]  -- только в скобках
UNION | UNION ALL | INTERSECT | EXCEPT
SELECT ...      -- множество 2
FROM ...
```

ТРЕБОВАНИЯ К ЗАПРОСАМ

- одинаковое число столбцов
- соответствующие столбцы — совместимых типов
- имена столбцов берутся из первого запроса

UNION: объединение

w_country

name	pop
France	67M
Spain	47M
Italy	60M
Andorra	0.08M

w_city

name	pop
Paris	2M
Madrid	3M
Rome	2.8M
Andorra	0.02M

ПРИМЕР

Все уникальные строки из двух множеств.

```
SELECT name AS geo_name, 'country' AS geo_type
FROM w_country
UNION
SELECT name AS geo_name, 'city' AS geo_type
FROM w_city;
```

Результат

geo_name	geo_type
France	country
Spain	country
Italy	country
Andorra	country
Paris	city
Madrid	city
Rome	city
Andorra	city

8 строк. Andorra — и страна, и город, но geo_type разный: строки не совпадают.

Без столбца geo_type строк было бы 7.

INTERSECT и EXCEPT

INTERSECT — есть в обоих множествах

ПРИМЕР

Найти страны и города с одинаковым названием.

```
SELECT name FROM w_city
INTERSECT
SELECT name FROM w_country;
```

Результат: Andorra

EXCEPT — есть в первой, нет во второй

ПРИМЕР

Найти страны, у которых нет города с таким же названием.

```
SELECT name FROM w_country
EXCEPT
SELECT name FROM w_city;
```

Результат: France, Spain, Italy

Обе операции возвращают уникальные строки. В EXCEPT порядок запросов важен.

w_country

name	pop
France	67M
Spain	47M
Italy	60M
Andorra	0.08M

w_city

name	pop
Paris	2M
Madrid	3M
Rome	2.8M
Andorra	0.02M

UNION ALL: объединение без удаления дублей

Логи кликов за два дня. Одинаковые строки возможны: повторная отправка события. Нам важно сохранить все события, в том числе дубликаты.

```
SELECT user_id, page, ts
FROM events_day1
UNION ALL
SELECT user_id, page, ts
FROM events_day2;
```

Результат: 8 строк — все события двух дней

С UNION было бы 7 строк: повторное событие «u1, /pdp, 11:05» пропало бы.

Зачем ALL? Нельзя терять повторные события — иначе собьём метрики, где нужны «сырые» клики.

events_day1

event_id	user_id	page	ts
e101	u1	/plp	10:01
e102	u1	/pdp	10:03
e103	u2	/plp	10:07
e104	u3	/home	10:10

events_day2

event_id	user_id	page	ts
e201	u2	/plp	11:02
e202	u1	/pdp	11:05
e203	u1	/pdp	11:05
e204	u3	/cart	11:06

INTERSECT ALL и EXCEPT ALL: с учётом кратности

INTERSECT ALL: что покупали и в январе, и в феврале

```
SELECT user_id, product_id
FROM order_items_jan
INTERSECT ALL
SELECT user_id, product_id
FROM order_items_feb;
```

order_items_jan

user_id	product_id
u1	A
u1	A
u1	B
u1	A

order_items_feb

user_id	product_id
u1	A
u1	A
u1	B
u1	C

Результат

user_id	product_id
u1	A
u1	A
u1	B

без ALL: A, B

EXCEPT ALL: поставки минус возвраты

```
SELECT sku, serial
FROM shipments
EXCEPT ALL
SELECT sku, serial
FROM returns;
```

shipments

sku	serial
A	s1
A	s1
A	s1
B	t1

returns

sku	serial
A	s1
B	t1

Результат

sku	serial
A	s1
A	s1

без ALL: 0 строк

UNION или UNION ALL — что выбрать

UNION	UNION ALL
убирает дубликаты (неявная дедупликация)	оставляет все строки
сортирует или хеширует результат, чтобы удалить дубли: дополнительная память и CPU	просто «склеивает» потоки без дополнительных этапов — работает быстрее

ПРИМЕР

Сумма продаж по категориям из двух источников.

```
SELECT category_id, SUM(amount) AS total_amount
FROM (
  SELECT category_id, amount FROM shop_sales_2024q1
  UNION ALL
  SELECT category_id, amount FROM shop_sales_2024q2
) t
GROUP BY category_id;
```

Когда в финале всё равно считаешь агрегаты, используй UNION ALL и агрегируй.

Если дубли нужно убрать — на финальных шагах, через DISTINCT или GROUP BY.

Сортировка в операциях над множествами

Вариант А: сортируем общий результат

```
SELECT name AS geo_name, 'country' AS geo_type
FROM w_country
UNION ALL
SELECT name AS geo_name, 'city' AS geo_type
FROM w_city
ORDER BY geo_name
LIMIT 50;
```

- сортировка применяется после объединения
- работают алиасы из первого запроса

Вариант Б: сортируем и ограничиваем один из запросов

```
(
  SELECT name AS geo_name, 'city' AS geo_type,
         population
  FROM w_city
  ORDER BY population DESC
  LIMIT 50
)
UNION ALL
SELECT name, 'country', population
FROM w_country;
```

- сортировка и лимит действуют только на первый запрос
- скобки обязательны
- без LIMIT внутренний ORDER BY может быть проигнорирован

Фильтрация и группировка

Фильтровать каждое множество — можно без скобок

```
SELECT ... FROM A WHERE ...
UNION ALL
SELECT ... FROM B WHERE ...;
```

Фильтровать общий результат — только через подзапрос

```
SELECT *
FROM (
  SELECT id, 'A' AS src FROM A
  UNION ALL
  SELECT id, 'B' AS src FROM B
) t
WHERE src = 'B'
```

Группировать общий результат — только снаружи

```
SELECT geo_type, COUNT(*) AS cnt
FROM (
  SELECT name AS geo_name, 'country' AS geo_type
  FROM w_country
  UNION ALL
  SELECT name AS geo_name, 'city' AS geo_type
  FROM w_city
) t
GROUP BY geo_type;
```

Результат: city — 4, country — 4

WHERE после последнего запроса без скобок относится только к этому запросу, а не ко всему результату.

Типовые ошибки

Ошибка	Плохо	Как надо

Неправильный порядок столбцов	`SELECT name, population FROM w_city`
	`UNION`
	`SELECT population, name FROM w_country`
		`SELECT name, population FROM w_city`
		`UNION`
		`SELECT name, population FROM w_country`

Несовместимые типы данных: timestamp и boolean	`SELECT id, created_at`
	`FROM flights_current`
	`UNION`
	`SELECT id, departed_flag`
	`FROM flights_archive`
		`SELECT id, created_at::text`
		`FROM flights_current`
		`UNION`
		`SELECT id, departed_flag::text`
		`FROM flights_archive`

Фильтр общего результата без подзапроса	`SELECT ... FROM A`
	`UNION ALL`
	`SELECT ... FROM B`
	`WHERE ...`
		`SELECT *`
		`FROM (SELECT ... FROM A`
		`UNION ALL`
		`SELECT ... FROM B) t`
		`WHERE ...`

ORDER BY по имени, которого нет в результате	`SELECT name AS geo_name FROM w_country`
	`UNION ALL`
	`SELECT name FROM w_city`
	`ORDER BY name`
		`SELECT name AS geo_name FROM w_country`
		`UNION ALL`
		`SELECT name FROM w_city`
		`ORDER BY geo_name`

ИТОГИ

Подведём итоги

Что нужно	Как пишется
Строки, у которых есть пара в другой таблице	`INNER JOIN … ON …`
Все строки одной таблицы, без пары — NULL	`LEFT JOIN, RIGHT JOIN`
Все строки обеих таблиц	`FULL JOIN`
Каждая строка с каждой	`CROSS JOIN`
Строки, у которых пары нет	`LEFT JOIN … WHERE … IS NULL`
Объединение, пересечение, разность результатов	`UNION, INTERSECT, EXCEPT`
То же с сохранением дублей	`UNION ALL, INTERSECT ALL, EXCEPT ALL`
Сортировка, отбор и группировка общего результата	`ORDER BY в конце; WHERE и GROUP BY — через подзапрос`